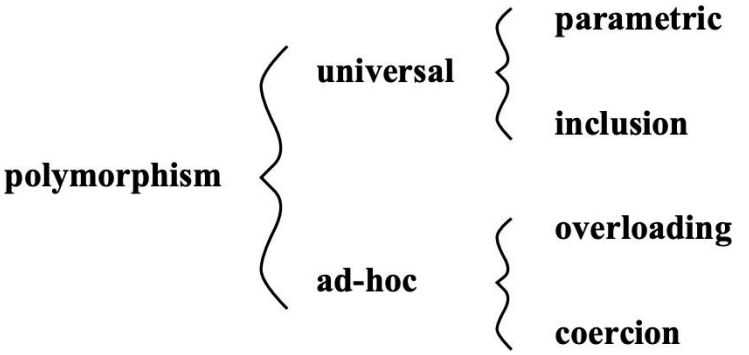
4.4. Полиморфизм

Полиморфизм

Полиморфизм — принцип, который позволяет объектам разных классов реагировать на один и тот же метод по-разному, что делает код гибким, переиспользуемым и модульным.

Виды полиморфизма в программировании:



В Go полиморфизм достигается через интерфейсы, которые позволяют переменной интерфейсного типа содержать значения любого типа, реализующего этот интерфейс.

```
package main
import "fmt"

type Dog interface {
 Bark()
}

type Dalmatian struct {
 DogType string
}

func (d Dalmatian) Bark() {
 fmt.Println("Dalmatian barking!!")
}

func MakeDogBark(d Dog) {
  d.Bark()
}

func main() {
   d := Dalmatian{"Jack"}
   MakeDogBark(d)
}
```

```
type Sortable interface {
   Len() int
   Less(int, int)
   bool Swap(int, int)
}

type Numbers []int

func (n Numbers) Len() int {
  return len(n)
}

func (n Numbers) Less(i, j int) bool {
   return n[i] < n[j]
}

func (n Numbers) Swap(i, j int) {
   n[i], n[j] = n[j], n[i]
}

func BubbleSort(s Sortable) { n := s.Len()
   for i := 0; i < n-1; i++ {
      for j := 0; j < n-i-1; j++ {
      if s.Less(j+1, j) {
            s.Swap(j, j+1)
         }
      }
   }
}

func main() {
   nums := Numbers{5, 3, 8, 1}
   BubbleSort(nums)
   fmt.Println(nums) // Вывод: [1 3 5 8]
}
```


MERION

```
type ILogger interface {
 Log(message string)
}

type Consolelogger struct{}

func (c Consolelogger) Log(message string) {
   fmt.Println(message)
}

type Filelogger struct { filename string
}

func (f Filelogger) Log(message string) {
   file, err := os.OpenFile(f.filename, os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644) if
   err != nil {
      fmt.Println("Failed to open file:", err) return
   }
   defer file.Close()

   if _, err := file.WriteString(message + "\n"); err != nil {
      fmt.Println("Failed to write to file:", err)
   }
}

func LogMessage(logger ILogger, message string) {
   logger.Log(message)
}

func main() {
   console := Consolelogger{}
   file := Filelogger{filename: "log.txt"}
    LogMessage(console, "Hello from console")
   LogMessage(file, "Hello from file")
}
```


MERION

Тестирование компонентов:

```
package main

import (
   "fmt" "testing"
)

type EmailSender interface {
   SendEmail(to, subject, body string) error
}

type RealEmailSender struct{}

func (r RealEmailSender) SendEmail(to, subject, body string) error {
   fmt.Println("Sending real email to:", to)
   return nil
}

type MockEmailSender struct { LastSentTo string
   LastSubject string LastBody string
}

func (m *MockEmailSender) SendEmail(to, subject, body string) error {
   m.LastSentTo = to
   m.LastSubject = subject m.LastBody = body return
   nil
}
```

```
func NotifyUser(emailSender EmailSender, user string, message string) {
   emailSender.SendEmail(user, "Notification", message)
}

func TestNotifyUser(t *testing.T) {
   mockSender := &MockEmailSender{}
   NotifyUser(mockSender, "user@example.com", "Hello")

   if mockSender.LastSentTo != "user@example.com" ||
     mockSender.LastSubject != "Notification" || mockSender.LastBody !=
     "Hello" {
      t.Error("Email not sent correctly")
   }
}

func main() {
   realSender := MockEmailSender{}
   NotifyUser(realSender, "user@example.com", "Hello")

   TestNotifyUser(&testing.T{})
}
```

Полиморфизм — это мощный инструмент, который делает программы более гибкими и удобными.

Универсальный, обеспечивает безопасность типов и повторное использование кода через общие интерфейсы.